QuakeWatch

CSCA 5028 Final Project Report: earthquake data collector, analyzer and search app

1. Purpose

QuakeWatch collects earthquake data from the public USGS API, stores it, analyzes it per region and shows the results on a small web site. USGS already publishes the data, but only as raw feeds that cover a short time window. QuakeWatch keeps its own history in a database, so a user can type a place (for example "Alaska") and see recent earthquakes there, or open the region report to compare activity between regions. Target users are people living in active areas, travelers and students who want a simple view without a map-heavy tool.

2. System requirements

Functional requirements

- Collect earthquakes from the USGS GeoJSON feed on a schedule (default every 15 minutes) without creating duplicates.
- Store each earthquake (id, magnitude, place, region, time, coordinates, depth, link) in a relational database.
- After each collection, compute per-region stats: number of earthquakes, max and average magnitude, latest time.
- Web page with a search form that echoes the user input and lists matching earthquakes (by place or region).
- Report page with the region stats.
- JSON API for search and stats, so other services can use the data.
- Health check and metrics endpoints for monitoring.

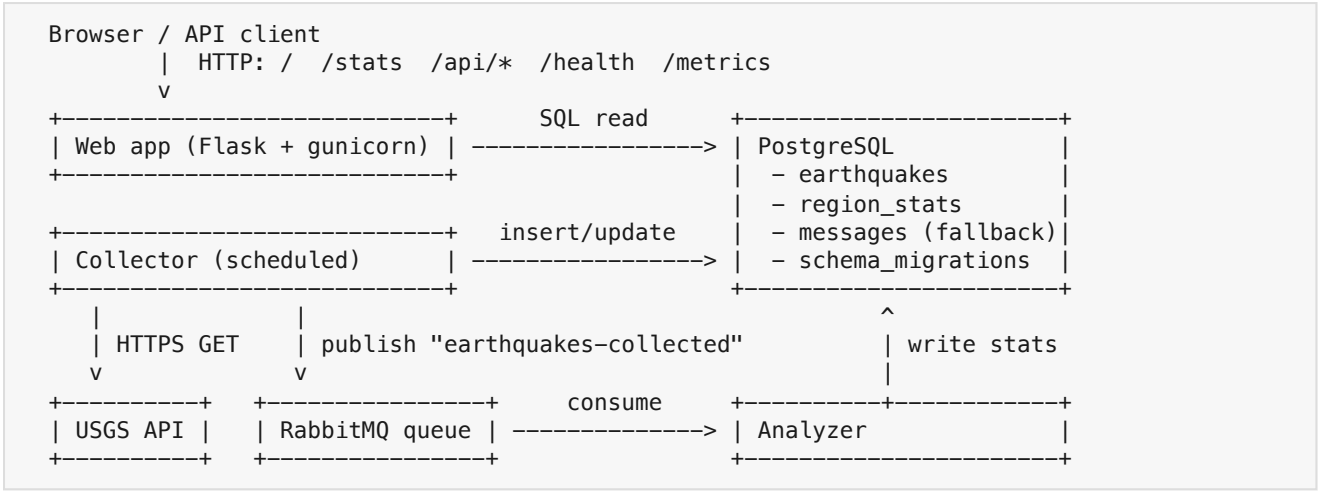
Non-functional requirements

- **Reliability:** a failed USGS call must not break the app. The worker logs the error, counts it in metrics and retries on the next tick. An event is only marked done after the analyzer finishes, so a crash doesn't lose it.
- **Loose coupling:** collector and analyzer communicate only through a message queue, and the web app only reads from the database. Each part can be changed or scaled on its own.
- **Security:** user input is escaped by the template engine and SQL uses bound parameters (no string building).
- **Performance:** search uses an index on `occurred_at` and a row limit. The data size is small (a few hundred rows per day), so one small instance is enough.
- **Cost:** runs on free hosting tiers.

Software and runtime requirements

Item	Version / choice
Language	Python 3.12
Web framework	Flask 3, served by gunicorn
Database	PostgreSQL 14+ in production and CI; SQLite for local work and unit tests
Messaging	RabbitMQ 3.10+ (via pika); falls back to a database queue table when `RABBIT_URL` is not set
Monitoring	prometheus-client (`/metrics`), `/health`, structured logs
Testing	pytest, responses (HTTP mocking), unittest.mock
CI/CD	GitHub Actions, Render deploy hook
Hosting	Render web service (Docker) and Render PostgreSQL

3. Architecture



The worker process runs the collector on a schedule and hands queued events to the analyzer. In production on the free tier the worker runs as a background thread inside the web process (`RUN_WORKER=true`), because a separate worker service is not free. The code also allows it to run as its own process (`python -m quakewatch.worker`).

4. Design decisions

Decision	Reason
USGS GeoJSON feed as data source	Public, no API key, updated every minute, stable format. Good for a scheduled collector.
Relational database (PostgreSQL)	The data is structured and the analysis is a GROUP BY query. Postgres is free on Render and in CI. SQLite is used locally so the project runs with no setup.
Plain SQL migrations with a small runner	Five migration files, tracked in `schema_migrations`. Easier to read than an ORM migration tool for a project this size. One placeholder handles the auto-increment difference between SQLite and Postgres.
Event collaboration between collector and analyzer	The collector doesn't need to know about the analyzer. It publishes an event and the analyzer reacts. New consumers (for example an alert service for large earthquakes) can be added without changing the collector.
RabbitMQ with a database queue fallback	RabbitMQ is used when available (CI, local docker compose). The free hosting has no managed RabbitMQ, so the same `MessageQueue` interface also has a database-table version. Both use manual acknowledgement: a message is only removed after the handler succeeds.
Analyzer recomputes all stats each run	Simple and always correct, and fast at this data size. If data grows, it can switch to incremental updates.
Region parsed from the place text	USGS has no region field. The text after the last comma ("..., Alaska") works for most rows. State codes like "CA" are mapped to full names.
Server-rendered pages plus a JSON API	Server-rendered HTML keeps the front end small and safe (auto escaping). The JSON API provides REST collaboration for other clients.
Prometheus metrics and health check	Metrics show collected count, failures, last collection time, analyzer runs and request latency. `/health` also checks the database and is used by Render to restart the service.
Docker image for deployment	The same image runs locally and on Render, so the production environment matches what was tested.

5. Rubric coverage

Rubric item	Where in the project
Web application (form and reporting)	`quakewatch/app.py`, search page `/` and region report `/stats`
Data collection	`quakewatch/collector.py`
Data analyzer	`quakewatch/analyzer.py`
Unit tests	`tests/unit/` (26 tests)
Data persistence	PostgreSQL / SQLite, `quakewatch/db.py`, `migrations/`
REST collaboration / API endpoint	`/api/earthquakes`, `/api/stats`; collector calls the USGS REST API
Product environment	`Dockerfile`, `render.yaml` (web service and Postgres on Render)
Integration tests	`tests/integration/test_pipeline.py`: real HTTP feed server, Postgres, RabbitMQ and the web app on a real port
Mock objects / test doubles	`responses` mocks the USGS API, `unittest.mock` in worker tests, fake publisher in collector tests
Continuous integration	`.github/workflows/ci.yml`, job `test` with Postgres and RabbitMQ services
Production monitoring	`/metrics` (Prometheus), `/health`, logs
Event collaboration / messaging	`quakewatch/messaging.py`, event `earthquakes-collected`
Continuous delivery	Job `deploy` in the CI workflow calls the Render deploy hook after tests pass on `main`

6. Testing

Unit tests run on SQLite with the USGS API mocked, so they are fast and need no network. They cover parsing, saving without duplicates, event publishing, region parsing, the analyzer, the database queue (order, single delivery, message kept when the handler fails), the worker schedule and retry, and every web endpoint, including escaping of user input. The integration test runs the whole flow (collect, publish, consume, analyze, serve over HTTP) against real Postgres and RabbitMQ when `DATABASE_URL` and `RABBIT_URL` are set, as they are in CI. I also ran the app with real USGS data: one collection saved 291 earthquakes, and the analyzer produced stats for 24 regions.

7. Limitations and next steps

- On the free tier the worker runs inside the web process, and Render puts free services to sleep after inactivity, so collection pauses while the site is idle. A paid worker or cron service would fix this.
- Region parsing from text is approximate for offshore places.
- Next features: alerts for large earthquakes as a second event consumer, and charts of daily counts per region.